Annex XIV — Scoped Worlds

Grouped Contexts, Publisher Orchestration, and Governed Local Actuation

Version 1.1 — 7 August 2026 (v1.0: 26 July 2026) · Part of the Optirando™ Canon · Status: Defensive Publication / Prior-Art Disclosure

Annex number provisional — assign per Canon sequence.

Document status: Normative unless explicitly marked as informative.

Applies to: the extension of the Public Handshake from single web presences to publisher-defined grouped contexts over physical and mixed environments (shops, exhibitions, trade-fair stands, production lines, buildings, smart homes, machine parks); the Scope Graph and its manifests; selection-scoped context loading and declared loading strategies; the Publisher Orchestrator, orchestrator-offline operation, session adoption, and multi-orchestrator arbitration; identity floors on the bidirectional path; and governed local actuation through Execution Providers under capability mapping.

XIV.0 Status, Scope, and Relationship to the Canon

This Annex extends the Public Handshake of Annex IX from a single contract-bound web presence to **publisher-defined grouped contexts**: one entry — one scan, one identifier, one designation — opens a governed scope containing many objects, whose individual contexts are loaded only upon selection, and within which declared actions may reach into the physical world through locally mapped Execution Providers.

This Annex does not redefine wire formats, cryptographic primitives, transport, handshake formation, rendering, pointer semantics, or execution semantics. It depends on, reuses, and does not weaken:

- **Annex VII** — the artifact and manifest machinery, publisher attestation (P0/P1/P2), the grant model (delivery and preparation rights only; no-expiry-until-revoke; unilateral silent revocation), and profile formation;
- **Annex IX** — the initiation layer (entry exclusively per IX.3.1; the code is an identifier and invitation, never authority), Hash-Pinned Consent, the two-object contract model, the change classes, anti-rollback versioning, the identity lock at formation (IX.3.3), the escalation rule (IX.3.7), and — decisively — the WR Trusted Execution Domain and the Finalizer execution lifecycle (IX.16–IX.23), of which the actuation model of this Annex (§XIV.8) is an instantiation, not a variant. This Annex takes up the research direction

reserved in IX.14: general capability declaration for non-web providers under the same declare-bind-limit-consent-block pattern;
- **Annex X** — WR Guard and capability-governed information flow on every external path;
- **Annex XI** — the WR Pointer as the designation layer (instrument catalog §XI.4.1 including identifier scan, proximity designation, and terminal/industrial input; the Unmapped Target Fallback of §XI.5; origin classes I4; explicit context bounding I8);
- **Annex III** — PoAC™ for every capability admission, including the local admissions of §XIV.8.

The core philosophy of the Canon applies without exception: **the system prepares, the human authorizes, the system records cryptographic proof.** Nothing in this Annex creates, implies, or pre-satisfies any execution authority.

XIV.0.1 Claims Discipline (Normative)

The claims discipline of Annex IX §IX.0.1 applies verbatim. Security statements in this Annex are either properties **by construction** of the specified mechanism, or **objectives** requiring implementation, testing, analysis, and audit before they may be claimed (§XIV.10). Nothing herein SHALL be represented as a certified or formally verified system.

XIV.0.2 Positioning (Normative)

The Optirando™ system is **account-bound secure automation for critical environments**. It is neither an anonymity system nor an identification system: identity disclosure is a declared, consented capability with a defined floor (§XIV.7). Toward Optirando™ infrastructure the operator is account-bound; toward publishers the system is **identity-minimizing by default** — publishers observe a pairwise, opaque subject binding, never a global identifier — and identifying only where a declared capability's identity floor requires it. Marketing and documentation derived from this Annex SHALL preserve this distinction and SHALL NOT describe the system as a privacy or anonymity product.

XIV.1 Motivation and Problem Statement (Informative)

Annex IX makes one publisher's web presence safe to enter for an operator who has never met that publisher. The physical world poses the same problem at a different granularity: an operator walks into a shop, an exhibition, a trade fair, a production hall — an environment containing hundreds of objects, each of which could carry context, actions, and consequences. Three naïve designs fail:

1. **One handshake per object** — unusable. A machine park with four hundred assets cannot demand four hundred scans, four hundred consents.
2. **One handshake delivering everything** — untenable. Transferring the complete context of an electronics store because someone scanned the entry code is wasteful, slow, and a profiling surface.
3. **Publisher must be live** — fragile. If the publisher's own coordination system must be online for a visitor to scan the poster at the entrance, availability of the entry path is coupled to the weakest server in the building.

The resolution of this Annex: the publisher declares a **Scope** — the shop, the exhibition, the line — as one governed context with one entry. Objects inside it are declared but not delivered; their contexts load **upon selection**, and only theirs. Everything statically declarable is signed into the repository in advance, so the entry path works whenever the operator and the Optirando™ repository can reach each other — the publisher's own orchestrator may be offline and adopt the session later. And where a declared action must touch the physical world — a light, a gate, a machine — the repository knows only the **semantic capability**; the binding to real endpoints happens exclusively on-site, inside the publisher's own infrastructure.

XIV.1.1 Worked Example (Informative) — The Many-Robot Floor

A production hall operates dozens of mobile robots, stationary cells, and autonomous transport systems. Under classical control, each carries its own panel, HMI, or terminal; an operator must know which unit, which interface, which addressing scheme, which credential. As the number of autonomous systems grows, the bottleneck is not their capability — it is **addressing and interface diversity**: coordinating many systems means juggling many control surfaces.

Under this Annex, the hall is one Scope — and its Scope Manifest is, in effect, a **signed capability catalog**: the publisher declares, for every unit separately, that unit's pre-defined command set — pause, resume, send to dock, release next job, hand over payload — each entry a declared capability with typed parameters, consequence class, identity floor, and `observed_effect` discipline (§XIV.8.2). Nothing shown at runtime is composed at runtime: every command the operator will ever see exists in the catalog before the first scan. An org-verified operator (ID2, §XIV.7) wearing smart glasses enters once. The glasses carry the complete WR Pointer of Annex XI — instrument availability is a hardware fact, not a device-class property (I9): **pointing with the finger at a robot** is a hand-ray/spatial-gesture designation within the live visual field (§XI.4.1), and the confirmed designation is the bounded context (I8). At that moment, and only then, exactly that unit's catalog entry loads — its declared context, its declared command set, and, **by reference from the handshake declaration, the complete automation authorizations behind them**: WCR-recording-comparable declared chains, hash-identified in the Scope Manifest and verified before any use (§XIV.5.1) — and the Augmented Overlay presents exactly these pre-defined commands with their origin classes: the menu is the context-filtered view of the catalog (Annex XI §XI.7), never a free surface. Invocation runs the unchanged path: client-generated preview, consent at the declared class — consequential commands stepping up to hardware-backed consent (C2/C3) where policy requires — the site's Execution Provider as Finalizer, witnessed transition, sealed receipt (§XIV.8.3).

The manageability claim, stated precisely: **the operator never addresses; the operator designates.** Selecting among fifty autonomous systems collapses from interface knowledge to pointing at the one that is meant — while nothing of the governance thins out: which actions exist is declared, whether they run is consented, and what happened is receipted. Complex multi-system scenarios — re-route this transporter, pause that cell, then release the queued job over there — become sequences of designations and declared invocations under one evidence chain; where several operators coordinate over the same Scope, the real-time coordination contexts of the Canon apply by reference. Functional safety remains untouched by construction: the WR path commands only within declared capabilities, and emergency stops, interlocks, and plant safety systems remain the device and plant domain (§XIV.10.4).

XIV.1.2 Worked Example (Informative) — The Electronics Store

A retail Scope shows the same architecture at consumer granularity. The store publishes one entry — a WR Code® at the door or per shelf — as an open public Scope; a customer enters at ID0, pairwise-pseudonymous (§XIV.7). The Scope index loads at entry; the shelves are child Scopes; the products are members. Pointing the smartphone camera or smart glasses at a product — or scanning its shelf tag (identifier scan, §XI.4.1) — loads exactly that product's declared context: specification, media, live price and stock through declared slots (Annex IX §IX.6.4), and its catalog entries — compare, reserve, order — as declared capabilities. **The customer never loads the store; the customer loads what interests them** (§XIV.5). Because the read path resolves locally and selection telemetry does not exist undeclared (§XIV.5.3–5.4), browsing the Scope is as silent toward the publisher as walking the aisles. Identity rises only where consequence begins: reserving or ordering carries the declared identity floor and consent class of that capability — and where an existing customer account comes into play, the login-bound binding of the Canon applies by reference, never a widening of the public Scope itself.

XIV.1.3 Worked Example (Informative) — The Smart Home

A household publishes its home as a private Scope: rooms as child Scopes, devices as members, each with its pre-defined command set in the signed catalog — light on/off and dim, blinds, playback volume, heating setpoint within declared bounds. Wearing smart glasses, a resident points a finger at the lamp, the speaker, or the blind (hand-ray designation in the live visual field, §XI.4.1); the designated device's catalog entry loads, and the overlay offers exactly its declared commands. Low-consequence commands run at their declared floor (C0/C1); the front-door lock and the garage gate are declared at higher consequence classes and step up accordingly (§XIV.8.2) — the difference between "dim the light" and "open the gate" is a difference of declaration, not of trust in the moment. On the actuation side, the repository knows only the semantics; the mapping of "turn on light" to the actual device lives in the household's local Execution Provider — **for example a Home Assistant instance**, equally a Node-RED, OpenHAB, ioBroker, or KNX deployment (§XIV.8.3): the provider is the local Finalizer host, the mapping a local admission, and nothing of addresses, integrations, or network topology ever appears in any repository artifact (§XIV.8.1, §XIV.8.4). Guests receive the same Scope under tighter inherited constraints — the ratchet of §XIV.4.3 as a household feature.

XIV.2 Definitions

Terms defined in Annexes II–XII apply as defined there. This Annex adds:

Scope. A publisher-declared, governed context boundary containing member objects and/or child Scopes. A Scope is simultaneously a *loading unit* (what one entry opens) and a *governance boundary* (where permissions, data-access ceilings, and identity floors are declared). The single-object Public Handshake of Annex IX is the degenerate Scope of depth zero.

Scope Graph. The publisher's declared hierarchy of Scopes and objects: Scope → child Scope → object, acyclic, with permission inheritance per §XIV.4.3.

Scope Manifest. The versioned, signed content object of a Scope, produced and governed under the Annex VII artifact machinery and the Annex IX manifest discipline (monotonic versioning, freshness, anti-rollback): member index, per-member context references (by SHA-256 digest), declared capabilities with their schemas (§XIV.8.2), the declared Loading Strategy (§XIV.5.2), declared telemetry semantics (§XIV.5.4), and identity floors (§XIV.7).

Scope Entry. Formation of a Public Handshake whose Handshake Contract binds a Scope (rather than a single web presence). Entry follows Annex IX §IX.3 unchanged: capture methods, verification chain, Hash-Pinned Consent, identity lock. Entry requires an operator account and reachability of the Optirando™ repository (§XIV.3.2).

Selection-Scoped Context Loading. The loading discipline of this Annex: designating a member object loads exactly that member's declared context — hash-verified against the Scope Manifest — and nothing else (§XIV.5). The load boundary is defined by the declaration, not by a client heuristic.

Publisher Orchestrator. The publisher-side runtime entity that manages the live half of a Scope: session adoption, bidirectional communication, live capability negotiation, and operational state. The orchestrator is distinct from every entry surface (website, poster, code, terminal), which holds zero authority (§XIV.6.1).

Session Epoch. A monotonic counter on a Scope session. Epoch transitions mark adoption, release, identity-floor escalation, and orchestrator handover; every transition is a signed, operator-visible event in the evidence chain (§XIV.6.3).

Orchestrator Set / Leadership Record. The publisher's declared set of orchestrators authorized for a Scope, with the signed record designating which orchestrator currently holds live authority for which partition (§XIV.6.4).

Execution Provider. A local system (home-automation runtime, industrial gateway, PLC connector, robotics controller, or equivalent) that implements declared semantic capabilities against concrete local endpoints. Architecturally, an Execution Provider is a **Finalizer host** in the sense of Annex IX §IX.8.4 (§XIV.8.3).

Capability Mapping. The exclusively local, locally admitted binding of a declared semantic capability to a concrete endpoint, device, or command (§XIV.8.3).

Local Capability Registry. The site-local registry of admitted Capability Mappings. Toward any remote party it answers availability at declaration granularity only ("capability X: available / unavailable"), never topology (§XIV.8.4).

Identity Class / Identity Floor. The four-step disclosure ladder of §XIV.7 and the minimum class a declared capability requires.

XIV.3 The Three-Party Availability Model

XIV.3.1 The Parties (Normative)

Every Scope session involves exactly three parties with distinct availability requirements:

1. **The operator's client** — requires an Optirando™ account and, for entry and for every first load of any artifact, network reachability of the repository. The WR system is account-bound without exception; there is no anonymous or accountless entry path. Offline operation exists exclusively over artifacts already loaded and verified (§XIV.3.3).
2. **The Optirando™ repository and relay** — the delivery source of everything statically declared: Scope Manifests, member contexts, templates, WCR recordings, capability schemas, LoRA adapters, and revocation state. Repository reachability is a precondition of entry and of every context load.
3. **The Publisher Orchestrator** — MAY be offline at entry and at any later time. Orchestrator reachability is a precondition of exactly the live capability classes of §XIV.3.2, and of nothing else.

XIV.3.2 The Static/Live Boundary (Normative)

The boundary between what works without the orchestrator and what requires it is **class-based, not feature-enumerated**:

- **Statically declared capabilities** — everything the publisher has declared, signed, and published into the repository in advance: member contexts, WR Pointer actions and their PAMs, adapters, declared automations up to and including locally finalized actuation (§XIV.8), and read-class interaction. These are available whenever operator and repository can reach each other, regardless of orchestrator state. Their authority derives from the repository-verified declaration and from operator consent — never from orchestrator liveness.
- **Live-negotiated capabilities** — everything requiring a decision, a message, or a state only the orchestrator holds: bidirectional messaging, P2P sessions, support sessions, live status beyond declared slots, and any capability the Scope Manifest marks `live`. These are unavailable while the orchestrator is offline and activate automatically upon adoption (§XIV.6.3). Their unavailability is presented as such — never simulated, never queued into silent later execution of consequential actions.

New capability classes sort themselves into this boundary by their declaration (`static` / `live` in the capability schema, §XIV.8.2); the boundary itself never moves.

XIV.3.3 Offline Semantics (Normative)

Offline operation is **cache, never entry**: a client that already holds verified artifacts MAY continue local, non-consequential use of them under the Annex IX staleness rules (last verified state, visibly marked stale, automations suspended pending freshness). Entry, first loads, revocation checks, and every consequential action requiring freshness fail closed without repository reachability. Because the repository must be reachable for entry and loading anyway, **revocation is checked against the repository** — revocation propagation is therefore independent of orchestrator liveness by construction.

XIV.4 The Scope Graph — Grouped Contexts

XIV.4.1 Publisher-Defined, Never Operator-Defined (Normative)

The Scope Graph is declared exclusively by the publisher: which objects belong together, which contexts are shared, which capabilities and ceilings apply. Operators navigate the graph; they never restructure it. Governance remains entirely with the publisher — and the publisher's declaration remains entirely subject to the verification chain.

XIV.4.2 One Entry, One Scope (Normative)

A Scope Entry (one scan, one identifier, one designation) opens exactly one declared Scope. The consent preview at formation presents the **Scope**, per the Annex IX preview contract: its identity, its ceiling (the Scope Envelope of the Handshake Contract), its declared capability classes including any actuation classes, its Loading Strategy, its telemetry declaration, and its identity floors — never a flattened enumeration of every member object. Members are what the Scope *contains*; the envelope is what the operator *consents to*.

XIV.4.3 Inheritance and the Ratchet (Normative)

Permissions, data-access ceilings, consent-class floors, and identity floors are declared at Scope level and inherited downward. A child Scope or member object MAY **tighten** any inherited constraint and SHALL NOT loosen any — the ratchet discipline of the Canon applied to the context hierarchy. An object belonging to multiple Scopes carries, in each session, the **intersection** of the effective permissions of the Scopes through which it was reached.

XIV.4.4 Scope Change Under a Standing Session (Normative)

Scope Manifests are versioned and freshness-bound per Annex IX §IX.4.2. Membership changes, capability additions, and ceiling expansions follow the Annex IX change classes: content movement within the envelope is silent; new capability classes, new actuation classes, ceiling expansion, and identity-floor increases are consent-class changes. A member object removed from the Scope becomes unavailable on the next freshness check; its already-loaded context is marked withdrawn.

XIV.5 Selection-Scoped Context Loading

XIV.5.1 The Discipline (Normative)

Designating a member object — through any instrument of Annex XI §XI.4.1 — loads exactly that member's declared context **and the execution artifacts its declared capabilities reference**: the automation authorizations behind the catalog entries — WCR recordings and declared chains in the sense of Annex IX §IX.8.1 and §IX.21.5 — are hash-identified in the Scope Manifest and loaded, verified against it before processing (Annex IX §IX.4.4 unchanged), only upon selection. Loaded material is admitted into the verified local store and bounded per I8. Nothing loads because it *might* be selected; nothing is withheld that *was* selected and declared. The load boundary is the declaration. This is a governance property with a performance benefit, not a performance

optimization: the operator **receives** only what they selected; the publisher **delivers** only what they declared.

An object designated within a Scope for which the Scope Manifest declares no context is an **unmapped target**; the Unmapped Target Fallback of Annex XI §XI.5 applies verbatim, including its no-publisher-signal rule.

XIV.5.2 Declared Loading Strategies (Normative)

The Scope Manifest declares one Loading Strategy per Scope (child Scopes may tighten toward `lazy`):

- **`lazy`** (default) — the Scope index loads at entry; member contexts load upon selection only.
- **`preload`** — the Scope index plus an explicitly enumerated preload set load at entry (declared members marked `preload: true`). Suitable for constrained-connectivity environments (trade-fair floors, field sites).
- **`hybrid`** — `preload` for the enumerated set, `lazy` for the remainder.

A Loading Strategy is a declared property of the Scope, stated in the entry preview — never a client heuristic and never a publisher-side runtime decision.

XIV.5.3 Index Locality (Normative)

The Scope index (member list, designation anchors, capability references — no member content) SHALL be compact enough to load fully at entry under every strategy. Consequence by construction: **read-path selection resolves locally.** Browsing the Scope — designating, viewing declared context already loaded or loading it from the repository — produces no request to the publisher's own infrastructure.

XIV.5.4 Selection Telemetry (Normative)

Every designation is an interest signal. Therefore:

1. Default: **no selection telemetry.** Neither designation events, nor load events, nor dwell information reach the publisher or the orchestrator.
2. A publisher MAY declare selection telemetry in the Scope Manifest — typed, bounded, purpose-stated — as a declared data-access class inside the Scope Envelope; it is then consent-covered at entry like any other envelope element and remains subject to WR Guard (Annex X) on egress.
3. Undeclared telemetry is a deviation with hard stop (Annex IX §IX.6.5). An unmapped designation (§XIV.5.1) is never a declarable telemetry event.

§XIV.5.5 Execution Authorization Proof Chain and Catalog Commitment

This subsection makes normative how a Public Handshake conveys cryptographically verifiable permission to execute repository-hosted artifacts, how a publisher administers that permission at any scale, and how an arbitrarily large publisher address space is conveyed without being loaded. It extends §XIV.5.1 (hash-identified execution artifacts, loaded and verified at selection) and resolves the hash-tree layout item of §XIV.12 within the Annex VII machinery; nothing in it relaxes

§XIV.5.1, and the Scope Manifest discipline (Annex IX monotonic versioning, freshness, anti-rollback) continues to apply per Scope unchanged.

(1) The proof chain. Every executable capability offered through a Public Handshake MUST be reachable through the following unbroken chain, and a capability for which the chain does not close DOES NOT EXIST for the runtime — it is not disabled, degraded, or warned about; it is absent (no hidden path):

1. the publisher's DNS discovery record pins the publisher root key (identity plane);
2. the origin manifest, signed by the root key, confirms the origin binding (dual-channel publisher verification per Annex IX §IX.3.5);
3. a **Catalog Head** — signed, epoch-numbered, freshness-bounded (see (2)) — commits to the publisher's entire address space;
4. the **Scope Manifest** for the grouped context in question, proven against the Catalog Head by Merkle inclusion proof, lists the scope's artifacts by content hash together with their Execution Authorization Declarations (see (3));
5. the artifact itself, content-addressed, matching the listed hash, carried with its dual assurances (see (5)).

Loading an artifact is capability acquisition and is admitted under PoAC™ (Annex III): PoAC admission for repository-hosted execution artifacts IS the verification that this chain closes, performed at selection time. Execution of consequence-bearing actions remains governed by PoAE™ and the consent-class machinery of Annex IX under the consent class floor declared per (3); the ratchet discipline of §XIV.4.3 applies — organization policy and the operator may escalate above a floor, never below it.

(2) Two commitment planes. The DNS discovery record and the Catalog Head serve different planes and MUST NOT be conflated:

- *Identity plane (DNS):* static and slow-moving. The discovery record pins keys and identity. It is NEVER updated to reflect catalog content changes; a publisher revising artifacts daily performs no DNS writes.
- *Content plane (Catalog Head):* the live address space. The Catalog Head is a compact signed object containing at minimum: the catalog root (a Merkle root over all Scope Manifests and entry objects of the publisher), a strictly monotonic epoch number, issuance time, and a declared freshness window; it is signed by the publisher root key or by a delegated Catalog Signing Key (see (4)).

Anti-rollback is mandatory: a client that has observed epoch n MUST reject any Catalog Head with epoch $< n$ for the same publisher. A Catalog Head past its freshness window degrades under the static/live model of §XIV.3.2 and the Annex IX staleness rules — cached scopes remain usable as visibly stale static content, but no NEW authorization-bearing capability may be admitted from a stale head. A 32-byte catalog root commits an address space of any size; the general address space of a Public Handshake transfers as this commitment only, never as content.

(3) The Execution Authorization Declaration. Each executable artifact listed in a Scope Manifest carries an Execution Authorization Declaration with at minimum the following fields:

- `artifact` — content hash of the artifact;

- `class` — the artifact class: pointer AI automation capability, adapter (e.g. safetensors LoRA), WCR recording, preparation view, recognition package, or a further class declared per §XIV.13;
- `scope_ref` — the scope (grouped context) the declaration is bound to; it is valid ONLY inside its scope;
- `consent_class_floor` — the minimum consent class for execution (C0–C4 per Annex IX §IX.18; ratchet: upward-only escalation);
- `identity_floor` — the minimum identity class ID0–ID3 (§XIV.7) required to load and to execute (the two MAY differ; execution never below load);
- `constraints` — at minimum a runtime version range; optionally an expiry;
- `epoch` — the catalog epoch the declaration was issued under;
- the publisher signature (root or delegated Catalog Signing Key).

Terminology note (normative): the Execution Authorization Declaration is an *authorization-to-execute* record and is distinct from the grant model of Annex VII (delivery and preparation rights); it modifies nothing in Annex VII and composes with it. Declarations are part of, or inseparably bound to, the Scope Manifest and are therefore covered by the catalog commitment; they are loaded and verified at selection together with the manifest, per §XIV.5.1. A verified declaration states what the publisher authorizes; it never overrides operator consent, organization policy, or WR Guard™ (Annex X) judgment — those may only narrow it.

(4) Delegation and administration. The publisher root key remains cold. Day-to-day catalog administration uses a **Catalog Signing Key** authorized by a single root-signed delegation record declaring: the delegate key, the delegated authority (catalog signing only), and the epoch range or revocation condition. Compromise of the Catalog Signing Key is remedied by an epoch bump plus a new delegation, without touching DNS. Administration — creating scopes, listing artifacts, issuing declarations, publishing Catalog Heads — is performed exclusively through the Publisher Orchestrator; entry surfaces remain zero-authority per §XIV.6.1. (The Publisher Console of the WR Entries annex is a user interface over exactly this administration model and introduces no additional authority.)

(5) Repository role and dual assurance. The Optirando™ repository (§XIV.3.1(2)) is the delivery source, and a CARRIER, never a trust anchor. Normatively:

- the publisher's repository namespace is bound to the publisher identity; upload authorization uses the same publisher identity and attestation chain that anchors the handshake;
- every artifact carries two independent assurances: the **publisher signature** (proves authorization — the chain of (1)) and the **platform ingest countersignature** (proves hygiene — normalization on ingest and Scam Watchdog™/WR Guard scanning per the repository discipline);
- a client MUST require both; neither substitutes the other. A hygienic artifact without a closing publisher chain is unauthorized; a publisher-signed artifact without ingest countersignature is unvetted. Either condition alone means the capability does not exist for the runtime;
- every published item is additionally publicly resolvable for audit (public auditability is specified in the WR Entries annex and applies to all catalog objects of this subsection).

Repository unavailability is governed by §XIV.3.3 unchanged: cache, never entry — an availability event, never an authority event.

(6) Load classes and storage discipline. To keep arbitrarily large address spaces from occupying operator storage, every artifact is assigned one of two load classes, declared in the Scope Manifest and composing with the Loading Strategies of §XIV.5.2:

- *entry-class substrate:* the minimal set required for entry, recognition, and designation to function at all — the Scope index of §XIV.5.3 and, where the respective annexes apply, the Recognition Package (Annex XV §XV.5.6) and the Entry Value Package (WR Entries annex). Entry-class content loads at entry and is intentionally small; the index-locality rule of §XIV.5.3 is its size discipline;
- *selection-class artifacts* (the default): everything else — pointer AI automation capabilities, adapters, WCR recordings, preparation views. Selection-class content loads ONLY upon selection, per §XIV.5.1.

Cache behavior is declared, never silent, and composes with §XIV.5.2: **pinned** scopes (retained until the operator unpins), **ephemeral** scopes (evicted on scope exit), publisher hints (size, priority, prefetch recommendation — advisory only, never binding), and **predictive preparation** as a declared strategy whose prefetching is visible and whose inferences are marked per the preparation rules. A conforming client holding zero cached content and the 32-byte catalog commitment is a fully valid holder of the complete Public Handshake address space.

XIV.6 The Publisher Orchestrator

XIV.6.1 Entry Surfaces Hold Zero Authority (Normative)

Websites, posters, printed codes, terminals, beacons, and every other entry surface are **distribution and designation surfaces** in the sense of Annex IX §IX.1.5: they carry identifiers, never authority. Everything verifiable — Scope Manifest, capability declarations, publisher keys, attestation — verifies against the repository and the dual-channel publisher verification of Annex IX §IX.3.5, never against the surface. A compromised entry surface is thereby reduced to denial of service and mis-distribution of codes that resolve to their own, correctly verified publisher — the residue Annex IX already scopes.

XIV.6.2 Orchestrator Responsibilities (Normative)

The Publisher Orchestrator holds exactly: adoption and release of standing Scope sessions; the live capability classes of §XIV.3.2; bidirectional and support-path communication under the identity floors of §XIV.7; and operational state beyond declared slots. It holds **no** authority over formation (repository-verified), none over static capability availability (declaration-derived), and none over the operator's evidence chain. The orchestrator is a *manager of the live half*, never a gatekeeper of the declared half.

XIV.6.3 Session Adoption (Normative)

When an orchestrator authorized for the Scope (§XIV.6.4) comes online against a standing session formed while it was offline:

1. **Verify, then adopt.** The orchestrator verifies the session's formation evidence (Handshake Contract reference, Scope Manifest version, consent records) against the repository. Verification failure means no adoption — never a repaired or partially adopted session.
2. **Epoch increment, visible transition.** Adoption increments the Session Epoch and is presented to the operator as a signed status transition in overlay and evidence chain (origin class (d), Annex XI I4). Live capabilities activate at the new epoch.
3. **No new handshake, no re-consent, no retroactive authority.** The standing relationship continues; nothing accumulated pre-adoption is re-consented, and nothing that occurred pre-adoption is retroactively legitimized or re-attributed — pre-adoption epochs remain marked as statically governed in the evidence chain.
4. **Release** (orchestrator going offline) is likewise an epoch event: live capabilities deactivate visibly; static capabilities continue unaffected.

XIV.6.4 Multi-Orchestrator Arbitration (Normative)

Where a publisher operates multiple orchestrators, silent races are excluded by construction:

1. The **Orchestrator Set** is declared in the Scope Manifest — enumerated orchestrator identities under the publisher root. An entity outside the set cannot adopt, regardless of credentials presented at runtime.
2. Live authority is held per the signed **Leadership Record** in the repository: exactly one leader per Scope partition. Default profile: single leader for the whole Scope. Enterprise profile: partitioned leadership along Scope-Graph boundaries (orchestrator A leads the shop floor, B the warehouse), partitions non-overlapping.
3. Handover is a Leadership Record update (publisher-signed, monotonic) followed by epoch increments on affected sessions. **Last-writer-wins, timeout-based self-election, and any adoption not derivable from the current Leadership Record are non-conforming** — an adoption attempt outside the record is a deviation with hard stop.

XIV.7 Identity Floors and the Bidirectional Path

XIV.7.1 Identity Classes (Normative)

Four identity classes, strictly ordered:

- **ID0 — pairwise-pseudonymous.** The default of every Scope Entry: the identity lock of Annex IX §IX.3.3 binds a stable subject binding into the Handshake Contract, and the fingerprint disclosed to the publisher is pairwise and opaque — stable within the relationship, uncorrelatable across publishers. Identity lock is not identity disclosure.
- **ID1 — account-verified.** Optirando™ attests "genuine account in good standing" without disclosing identity. (Anti-abuse floor for open public Scopes.)
- **ID2 — org-verified.** The operator is attested as a member of a named organization, without personal identification. (The standard enterprise, maintenance, and field-service floor.)

- **ID3 — person-identified.** Personal identity is disclosed under a declared verification profile.

XIV.7.2 Floors, Ratchet, Visibility (Normative)

1. Every capability declares its **Identity Floor** in its schema (§XIV.8.2); read-path browsing of a public Scope SHALL carry floor ID0. Publisher, organization policy, and operator may escalate above a floor; none may execute below it (ratchet).
2. Crossing to a higher class is an **explicit, visible consent event** and an epoch transition (§XIV.6.3(2)); it applies from that epoch forward and never retroactively re-identifies earlier epochs. An aborted escalation leaves the pseudonymous session undamaged.
3. **Symmetry on the bidirectional path.** Where a capability requires ID2/ID3 of the operator (support session for a machine, account-bound service), the publisher side is held to the same discipline: the acting publisher agent is org-verified against the publisher's attestation, and the session is verified through the handshake relationship itself (the BEAP™-verified "was it you?" pattern, Login-Bound section by reference). Identity requirement is mutual, never one-sided.
4. Escalation to full-context bidirectional classes remains governed by Annex IX §IX.3.7 (new consent event, nothing pre-satisfied).

XIV.8 Governed Local Actuation — Execution Providers and Capability Mapping

XIV.8.1 The Two-Sided Discipline (Normative)

Physical actuation is governed by one rule with two sides:

- **The repository knows the meaning, never the machine.** Repositories and Scope Manifests SHALL contain no endpoint URLs, no IP addresses, no local API descriptions, no credentials, no secrets, and no network topology. They declare exclusively **semantic capabilities** under typed schemas (§XIV.8.2). This is the third instance of one Canon discipline: identifiers-only slots (Login-Bound), taxonomy-only cloud abstraction (Annex III §7), and now semantics-only actuation declaration.
- **The site knows the machine, never more than declared.** The binding of a semantic capability to a concrete endpoint occurs exclusively on-site, in the Execution Provider, under local PoAC admission (§XIV.8.3). Infrastructure, networks, and devices remain fully local; the publisher-side declaration and the operator-side consent see the semantic capability only.

XIV.8.2 The Capability Schema (Normative)

A semantic capability is never a free verb phrase. Its declaration in the Scope Manifest SHALL carry, at minimum:

1. stable capability identifier and semantic version;
2. human-readable effect statement (basis of the client-generated preview, Annex IX §IX.5.3);
3. typed, bounded parameter schema (instruction-incapable grammars preferred, Annex IX §IX.8.6(4));

4. **consequence class** and the derived consent-class floor C0–C4 (Annex IX §IX.18) — "increase volume" and "open gate" are different declarations by construction;
5. **Transition Class** — `atomic`, `journaled`, or `observed_effect` per Annex IX §IX.21.3; physical-world effects are typically `observed_effect` and inherit its stricter consent-minimum policy;
6. **Identity Floor** (§XIV.7);
7. availability class `static` / `live` (§XIV.3.2);
8. validation rules and expected-state/post-state declaration for witnessing;
9. Finalizer binding by reference (§XIV.8.3);
10. revocation conditions.

For spatially anchored capabilities (AR, field service), the schema MAY additionally bind a **target descriptor** — the declared linkage between the capability and the designated physical object (member identity, spatial anchor class) — declared here as an extension point (§XIV.12).

XIV.8.3 The Execution Provider as Finalizer Host (Normative)

An Execution Provider is not a new execution model; it is the **local host of declared Finalizers** in the exact sense of Annex IX §IX.8.4 and §IX.16–IX.23:

1. Every consequential physical effect passes the five-phase lifecycle **Validate → Execute → Witness → Seal → Anchor** unchanged: capability and consent verification including Intent Hash, deterministic execution of the declared transition, pre-/post-state witnessing per Transition Class, sealed receipt into the operator's evidence chain, optional tiered anchoring (L/C/X).
2. **Capability Mapping is local Finalizer configuration** and is itself a capability acquisition: mapping capability X to endpoint Y is a PoAC admission event on-site (evidence-backed, reviewed, signed, revocable — propose, never silently apply). Who may map, and to what, is site policy under Annex III.
3. Provider independence is normative: the mapping interface binds any conforming provider — home-automation runtimes (e.g. Home Assistant, Node-RED, OpenHAB, ioBroker, KNX), industrial protocol gateways (e.g. OPC UA), PLC and robotics connectors, and custom local systems alike; the named products are examples, and no provider product is privileged or required by this Annex.
4. Compromise posture inherits Effect Binding (Annex IX §IX.8.8): a compromised path upstream of the Provider can at most fail to produce the agreed effect — never produce a non-agreed one — under the stated trust assumption of correct local Finalizer implementation (§XIV.10.3).

XIV.8.4 The Local Capability Registry (Normative)

The site maintains the Local Capability Registry of admitted mappings. Toward the orchestrator, the publisher, and every remote party it answers **at declaration granularity only**: capability available / unavailable. It SHALL NOT disclose provider identity, endpoint, protocol, address, or topology on any remote interface. Availability answers are informational and confer nothing (information without authority, Annex II §II.2.5).

XIV.8.5 Middleware Posture (Informative Deployment Guidance)

Every additional middleware layer is additional attack surface. Execution Providers are therefore optional, never required: a Scope without actuation is complete. Deployments — enterprise and training environments in particular — SHOULD minimize middleware and prefer few, auditable Providers over broad integration fabrics. Where Providers are deployed, they stand under the Canon like everything else: declared capabilities, local PoAC, no hidden path. Openness for extension is preserved precisely because the extension mechanism is governed.

XIV.9 Designation Integration (Normative, by Reference)

Designation of Scopes and members uses the WR Pointer unchanged: every instrument of Annex XI §XI.4.1 — cursor and touch, camera and live feeds, identifier scan, proximity designation, terminal/industrial input, declared WR Link activation — feeds the identical semantic sequence (designate → bounded context → resolve admitted capabilities → invoke under consent). Scope and member designation are trigger contexts in the sense of Annex XI §XI.7. This Annex adds no designation semantics; a genuinely new instrument class is added in Annex XI under its §XI.20, never here.

XIV.10 Security Model

XIV.10.1 Properties by Construction (Normative Claims)

- No entry without account and repository verification; no anonymous path exists (§XIV.3.1).
- No formation, extension, or upgrade of a Scope session through any entry surface; surfaces carry identifiers only (§XIV.6.1; Annex IX §IX.3.1).
- Static capability availability is independent of orchestrator liveness, and orchestrator unavailability can never silently downgrade, simulate, or defer consequential capabilities (§XIV.3.2).
- Revocation checking is independent of orchestrator liveness, because entry and loading already require repository reachability (§XIV.3.3).
- No member context loads without a prior hash match against the signed, monotonic, freshness-bound Scope Manifest; the load boundary is the declaration (§XIV.5.1).
- Read-path browsing produces no publisher-side signal, and no selection telemetry exists undeclared and unconsented (§XIV.5.3–5.4).
- Inherited constraints only ever tighten; multi-membership resolves to the intersection (§XIV.4.3).
- No adoption outside the declared Orchestrator Set and current Leadership Record; adoption is epoch-incrementing, operator-visible, and never retroactive (§XIV.6.3–6.4).
- Publishers observe a pairwise, opaque subject binding at ID0; no capability executes below its declared Identity Floor; escalation is visible, epoch-bound, and never retroactive (§XIV.7).

- No endpoint, address, credential, or topology exists in any repository artifact; the remote-visible surface of local infrastructure is capability availability at declaration granularity (§XIV.8.1, §XIV.8.4).
- Every physical effect passes a declared, locally admitted Finalizer under the five-phase lifecycle with Intent Hash binding and witnessed transitions; a compromised upstream path can at most withhold the agreed effect (§XIV.8.3; Annex IX §IX.8.8, §IX.17).

XIV.10.2 Objectives Pending Validation (Informative)

Correctness and completeness of `observed_effect` witnessing on heterogeneous device classes; robustness of the Local Capability Registry's non-disclosure boundary; behavior of adoption and leadership handover under network partition; scalability of Scope indices for very large member sets; spatial-anchor binding reliability for AR-class target descriptors; and the operational security of common Execution Provider platforms under the local Finalizer role. These require implementation, testing, adversarial analysis, and audit before being claimed.

XIV.10.3 Trust Assumptions (Informative)

The Annex VII/IX publisher root and attestation chain; Optirando™ repository integrity and the Assessment Record Store; platform integrity of the operator device; and — specific to this Annex — the correct, atomic-or-witnessed, replay-resistant local implementation of declared Finalizers by Execution Providers, and the site's own admission discipline over Capability Mappings. A malicious but correctly attested publisher declaring harmful-but-consented capabilities is bounded by the preview contract, consequence classes, and identity floors — not eliminated (Annex IX §IX.12.4 applies: the system proves who declares and what can execute, not that the publisher is virtuous).

XIV.10.4 Non-Goals (Normative)

This Annex does not guarantee physical safety properties of actuated devices (interlocks, functional safety, SIL ratings remain the domain of the respective device and plant safety systems); does not replace regulatory compliance of the deployment; and does not prove that a Scope's declared representation matches physical reality.

XIV.11 Relation to Prior Work and Claimed Contribution (Informative)

The constituent mechanisms have established neighbours and are not claimed as novel individually: QR-/NFC-triggered content and proximity marketing (one code, one URL — no verification chain, no contract, no consent governance); smart-home and building standards and hubs (device abstraction and local control — no per-operator consent binding, no evidence chain, no semantic-only remote surface); industrial companion specifications and digital-twin models (typed device semantics — descriptive, not consent-governed execution contracts); capability-based security (attenuated authority tokens — here composed with human consent minting and receipts); and service discovery protocols (availability announcement — here reduced to declaration-granularity answers under a non-disclosure boundary).

The claimed contribution is the governed composition: one verified entry opening a publisher-declared, ratchet-inheriting Scope Graph; selection as the load boundary with default-silent read paths; formation and static capability decoupled from publisher-side liveness with signed, epoch-visible session adoption and record-based orchestrator arbitration; a four-class identity ladder with per-capability floors, pairwise-pseudonymous default, and mutual identification on the bidirectional path; and physical actuation in which the remote world knows only semantics, the local site binds under its own admission discipline, and every effect is finalized, witnessed, and receipted under the unchanged Annex IX execution model. To the author's knowledge, no existing system composes these elements in this consent-bound form. Each element and the combination are hereby published as prior art.

XIV.12 Open Specification Items (Informative)

Scope Manifest schema and index format (member anchors, designation references, hash-tree layout within the Annex VII machinery); Loading Strategy declaration grammar and preload-set format; selection-telemetry declaration schema (types, bounds, purposes); Leadership Record format and handover protocol; adoption verification message flow; identity-class attestation profiles for ID1–ID3 and their evidence carriage; capability schema serialization; the Capability Mapping admission record format and the Local Capability Registry remote interface; target-descriptor formats for spatially anchored capabilities; partition-declaration grammar for enterprise leadership profiles.

XIV.13 Extension Points and Amendment Discipline

Declared extension points (additive only): new identity-class attestation profiles within the ID0–ID3 ladder; new Loading Strategies beyond `lazy`/`preload`/`hybrid`; target-descriptor classes for spatial/AR anchoring; capability-schema field additions; Leadership partition profiles; telemetry declaration classes. Amendments follow the Canon discipline: versioned, never silent, recorded in §XIV.14; changes to normative invariants require explicit identification and justification (Annex XI §XI.0 pattern).

License Notice

XIV.14 Version History

| Version | Date | Change |

|---|---|---|

| 1.0 | 2026-07-26 | Initial normative specification, with three worked examples as informative motivation (§XIV.1.1 many-robot floor via smart glasses, §XIV.1.2 electronics store, §XIV.1.3 smart home with Execution Provider actuation): three-party availability model (account-bound entry, repository as static delivery source, orchestrator-optional liveness) with class-based static/live boundary and cache-never-entry offline semantics; the Scope Graph as grouped contexts (publisher-defined, one-entry-one-Scope with envelope-level consent, ratchet-only inheritance, intersection

under multi-membership, change-class-governed evolution); selection-scoped context loading with declared Loading Strategies, local index resolution, default-silent read path, and declared-only telemetry; Publisher Orchestrator model (zero-authority entry surfaces, adoption with epoch-visible transitions and no retroactive authority, Orchestrator Set and Leadership Record arbitration excluding silent races); identity classes ID0–ID3 with per-capability floors, pairwise-pseudonymous default, epoch-bound non-retroactive escalation, and mutual identification on the bidirectional path; governed local actuation (semantics-only repository discipline, typed capability schemas with consequence class / Transition Class / Identity Floor, Execution Provider as local Finalizer host under the unchanged Annex IX five-phase lifecycle, Capability Mapping as local PoAC admission, Local Capability Registry with declaration-granularity non-disclosure, middleware-minimization deployment guidance); designation integration by reference to Annex XI; security model with by-construction properties, pending objectives, trust assumptions, and non-goals; prior-work relation and claimed contribution; open specification items and extension points. |

| 1.1 | 2026-08-07 | Added §XIV.5.5 (Execution Authorization Proof Chain and Catalog Commitment), extending §XIV.5.1 and resolving the hash-tree layout item of §XIV.12: unbroken proof chain from DNS-pinned identity to content-addressed artifact with chain-closure as PoAC admission (no hidden path); two-plane commitment model (static identity plane in DNS, live content plane as signed, epoch-numbered, freshness-bounded Catalog Head with mandatory anti-rollback; 32-byte root commits the full address space); Execution Authorization Declaration per artifact (class, scope binding, consent class floor C0–C4, identity floor ID0–ID3, constraints, epoch; distinct from and composing with the Annex VII grant model); cold root key with delegated Catalog Signing Key, administered exclusively through the Publisher Orchestrator; repository dual assurance (publisher signature = authorization, ingest countersignature = hygiene, both required); entry-class vs. selection-class load discipline composing with §XIV.5.2/§XIV.5.3 (pinned/ephemeral scopes, advisory hints, declared predictive preparation). Insertion-only; no existing invariant modified. WR Entries, the Entry Value Package, and the Publisher Console are specified in a standalone annex by reference. |

Scoped Worlds — One Scan Opens a Governed World

1 / 6

One entry, one Scope. Objects load on selection — never wholesale, never silently observed.

1 Enter one governed Scope

Entry (one scan)

Shop · Hall · Home

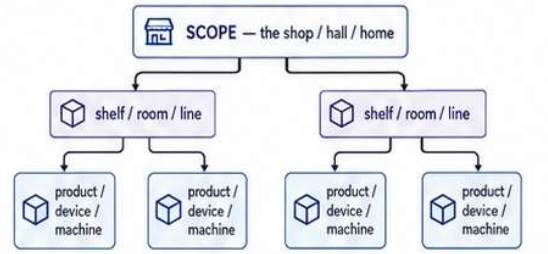
Scope Card

SCOPE
— the shop / hall / home

Scope ID	sc-7F9A-2C3D
Publisher	Acme Retail Ltd.
Root Signature	SIG-7E8B…42D1
Issued	2025-05-07 10:12Z
Depth	0 (root)

Valid & Verified

Scope Graph (example)



publisher-defined — never operator-restructured

one entry = one Scope

single object = Scope of depth zero

ratchet inheritance: children may only tighten

multi-membership → intersection of permissions

2 Signed capability catalog

declared per member, signed under the publisher root, hash-identified

Smart Light A19

Commands: pause · reserve · dim

typed params	yes
consequence class	low
identity floor	member
transition class	reversible
Member ID	m-11A3-9B2E…
Catalog Hash	H-90AF…C312

Conveyor Line 3

Commands: pause · inspect · start

typed params	yes
consequence class	medium
identity floor	member
transition class	controlled
Member ID	m-22D4-7C5F…
Catalog Hash	H-B712…E44D

Shelf Item B12

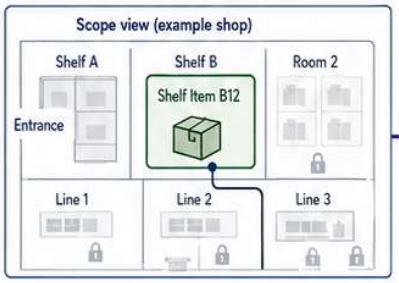
Commands: reserve · locate · compare

typed params	yes
consequence class	low
identity floor	member
transition class	reversible
Member ID	m-33F6-1D8A…
Catalog Hash	H-3C9E…7A18

Nothing shown at runtime is composed at runtime — every command exists in the catalog before the first scan.

menu = context-filtered view of the catalog — *never* a free surface

3 Selection-scoped loading



designate (point / scan / touch)

load exactly this member: context + referenced automation artifacts (WCR-class chains), hash-verified

Shelf Item B12

hash verified H-3C9E…7A18

everything else stays unloaded

- load boundary = the declaration
- index loads fully at entry — read path resolves locally
- no wholesale transfer of the whole store
- no selection telemetry unless declared + consented
- unmapped designation → local fallback, no publisher signal (Annex XI)

4 Local browse → selected load → declared invocation

Scan (enter the Scope) → Verify (signature & integrity) → Consent (operator consent) → Browse (local index) → Select (choose one member) → Load (hash-verified artifacts) → Invoke (issue declared commands) → Receipt (verifiable local receipt)

What This Guarantees

- one entry, one Scope
- only selected member loads
- verifiable local execution

What This Explicitly Excludes

- silent observation
- bulk preloading
- undeclared surfaces

★ *The operator receives only what they selected; the publisher delivers only what they declared.*

Three Parties, One Availability Model — The Orchestrator May Sleep

Entry needs account + repository. The publisher's orchestrator may be offline — and adopt later.

The Three Parties

1 Operator Client

account required, online for entry & first loads

REQUIRED

account · device · integrity

2 Optirando™ Repository

delivers everything statically declared

REQUIRED

- **Site Manifests** (pages, hashes, slots, WR Links)
- **Member Contexts** (objects, permissions, identity floors)
- **Signed Capability Catalogs** (commands, params, classes)
- **WCR Recordings** (semantic chains, capsules, receipts)
- **Automation Instructions** (workflows, triggers, finalizers)
- **Normalized Media** (images, docs, models)
- **Agent Orchestration Logic** (policies, routing, rules)
- **LoRA Adapters & Models** (safetensors, hashes)
- **Mappings & Connectors** (semantic↔endpoint)
- **Revocation Lists & Policies** (leases, epochs, validity)

3 Publisher Orchestrator

live half only

MAY BE OFFLINE

live services · messaging · support · P2P

- offline = cache, never entry
- revocation checks ride on the repository — orchestrator-independent by construction

The Static / Live Boundary

A STATIC — works while orchestrator sleeps	B LIVE — activates on adoption
member contexts & permissions	messaging (direct, group)
pointer actions (designations, selections)	P2P connections
adapters (LoRA, models, tools)	support sessions (live)
declared automations (triggers, workflows, finalizers)	live status beyond declared slots
WCR recordings & receipts	live telemetry (declared channels only)
normalized media (images, docs, models)	live alerts & presence
agent orchestration logic (policies, routing, rules)	ephemeral session state & locks
mappings & connectors (semantic↔ endpoints)	
locally finalized actuation (READ · TRANSMIT · CHANGE · PERSIST · EXECUTE)	

boundary is class-based (static/live in the declaration) — never a feature list

unavailability is shown, never simulated, never silently queued

Adoption & Leadership

Epoch 1 — formed offline (statically governed)

orchestrator verifies formation evidence (repo-signed artifacts, receipts)

Epoch 2 — ADOPTED (visible status transition, live capabilities activated)

Epoch 3 — released (live off, static continues)

Multiple orchestrators

Orchestrator Set + signed Leadership Record — one leader per partition

- no last-writer-wins
- no timeout self-election
- no adoption outside the record

nothing pre-adoption is retroactively legitimized

Form (repo-verified) → handshake formed from declared artifacts

Work (static) → use contexts, automations, adapters locally

Orchestrator online → publisher's orchestrator becomes available

Verify evidence → formation proof verified against repository

Adopt (epoch++) → adoption recorded, live boundary opens

Live on → live capabilities active per declaration

Release → live half deactivated

Static continues → statically declared capabilities remain

The orchestrator manages the live half — it is never a gatekeeper of the declared half.

Identity Floors ID0–ID3

Pointer selection unlocks governed interaction.
Identity disclosure appears only when a declared capability requires a higher floor.

1 Select an object with the WR Pointer

Shop shelf / product

Industrial machine / cabinet

Autonomous mobile robot

A) Adaptive WR Menu unlocks

- Inspect
- Reserve
- Pause
- Diagnostics
  - Status
  - Logs
  - Sensors
  - Events
- Route

Menu is context-aware and member-specific.

B) Load only this member

- context
- permission scope
- declared capabilities
- referenced automation artifacts

- selection-scoped loading
- permission scope resolved
- everything else stays unloaded

selection alone does not identify the person

The selected object becomes the active governed context.

2 Identity floors ID0–ID3

ID3 — person-identified
declared verification profile

ID2 — org-verified
member of org X
person undisclosed

ID1 — account-verified
genuine account attested
identity undisclosed

ID0 — pairwise-pseudonymous
default on entry
opaque per-publisher binding

identity lock ≠ identity disclosure

Capability floors

- browse selected object — Floor: ID0
- field-service control / diagnostics — Floor: ID2
- account-bound order / binding change — Floor: ID3 — login-bound to website

3 Login binding and epoch transition

1 Browse at ID0

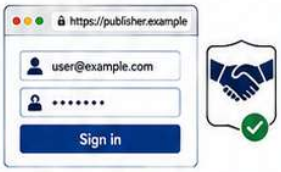
silent read path

2 Preview shows a higher floor

This action requires ID3.
Preview

consequence preview states the required floor

3 Login binding on the publisher website

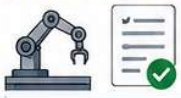


website login binds the higher-identity capability to the existing relationship.

4 Epoch++

- visible consent event
- applies forward only

5 Act at required floor



execute action at the declared floor

Read-path browsing never identifies.

No retroactive re-identification

No ad-hoc support claim as identity proof

Aborted escalation leaves the pseudonymous session intact.

1 **Enter** — default ID0
2 **Select** — point at member
3 **Menu** — adaptive options
4 **Load** — context + scope
5 **Preview** — floor stated
6 **Login-bind** — website session
7 **Act** — required floor
8 **Receipt** — verifiable record

What This Guarantees

- ✓ selection-scoped interaction
- ✓ per-capability floor disclosure
- ✓ forward-only escalation
- ✓ read path may remain pseudonymous

What This Excludes

- ✗ global forced identity
- ✗ retroactive re-identification
- ✗ object selection as identity disclosure
- ✗ support-as-identity-proof

The object unlocks the context; the capability states the floor; identity rises only when the declaration requires it.

Governed Local Actuation — The Cloud Knows the Meaning, Never the Machine

Repositories declare semantics. Sites map endpoints. Every effect is finalized, witnessed, receipted.

1 Two sides of one rule

REPOSITORY — knows the meaning

Semantic capability examples

turn on light	start machine	open gate
typed params	typed params	typed params
C0–C4	C0–C4	C0–C4
identity floor	identity floor	identity floor
transition class	transition class	transition class

pause robot	dim room
typed params	typed params
C0–C4	C0–C4
identity floor	identity floor
transition class	transition class

Never disclosed by the repository

- no URLs
- no credentials
- no IPs
- no topology
- no local APIs

SITE — knows the machine

Local plant / home network (example)

Lights Bus — PLC / Controller — Machine #12 — Gate Controller — Sensors

Capability Mapping (semantic → local endpoint)

start machine	local PLC connector (plc://line3/start)
open gate	gate controller (io://gate/open)
turn on light	lighting bus (knx://group/12/on)

PoAC admission note: mapping is proposed for admission — never silently applied.

mapping stays exclusively local

2 Execution Provider = local finalizer host

1. **Validate** — capability · consent · Intent Hash
2. **Execute** — declared transition
3. **Witness** — pre / post-state · observed effect
4. **Seal** — signed receipt
5. **Anchor** — L / C / X evidentiary only

Worked example (local actuation)

A. Select object (WR Pointer)

Machine #12
Type: CNC
Line 3
State: Idle

B. Adaptive WR Menu (from repo)

Inspect
Start
Pause
Open
...

C. Consent + preview

Class: C2 · Floor: Operator
Effect: Start machine
Intent Hash: 7f3a...8e21
Preview OK

D. Local execution provider

Exec Provider (Home Assistant)
Executes via PLC connector

E. Effect achieved locally

Machine state → Running
Observed at 10:15:32

Examples of execution providers (none privileged, none required)

Home Assistant · Node-RED · OpenHAB · ioBroker · KNX · OPC UA gateway · PLC / robotics connector

A Scope without actuation is complete — execution providers are optional. Minimize middleware.

3 The non-disclosure boundary

LOCAL SITE (plant / home)	REMOTE / UPSTREAM (interface)
Local mappings inside the boundary	NON-DISCLOSURE BOUNDARY
PLC connector (plc://...)	API response: capability X: available / unavailable
Lighting bus (knx://...)	never provider identity
Gate controller (io://...)	never endpoint
Machine API (opcua://...)	never protocol
...	never address
	never topology

A compromised upstream path can at most withhold the agreed effect.

It can never produce a non-agreed one (Effect Binding).

e-stops, interlocks, functional safety remain the plant's domain.

Legend

- local knowledge & execution (never disclosed)
- capability metadata only (semantic response)
- non-disclosure boundary

4 What the operator experiences

1 Declare	2 Sign	3 Select	4 Consent	5 Map	6 Finalize	7 Witness	8 Receipt
semantics only	capability catalog signed	object chosen with WR Pointer	floor + class preview	local admitted mapping	local effect executed	observed effect recorded	verifiable result
start machine, turn on light, open gate, ...			Class: C2 Floor: Operator Effect: Start Preview OK				

What this guarantees

- taxonomy-only cloud
- local-only endpoint knowledge
- declared effects only
- verifiable local receipt

What this explicitly excludes

- remote endpoint disclosure
- silent connector injection
- cloud-side machine control details
- undeclared actuation paths

End-to-end flow (eight steps)

1 Declare (semantics only) → 2 Sign (catalog signed) → 3 Select (WR Pointer) → 4 Consent (preview OK) → 5 Map (local mapping) → 6 Finalize (effect executed) → 7 Witness (effect recorded) → 8 Receipt (verifiable result)

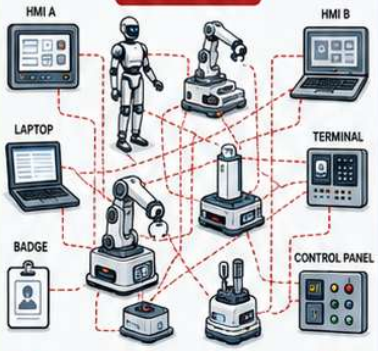
The third instance of one discipline: identifiers-only slots, taxonomy-only cloud, semantics-only actuation.

The Many-Robot Floor — Designate, Don't Address

Fifty autonomous systems. One pair of smart glasses. Pointing replaces addressing — governance stays whole.

Before / After

CLASSICAL



per-unit interfaces · addressing schemes · credentials · interface diversity bottleneck

SCOPED WORLD

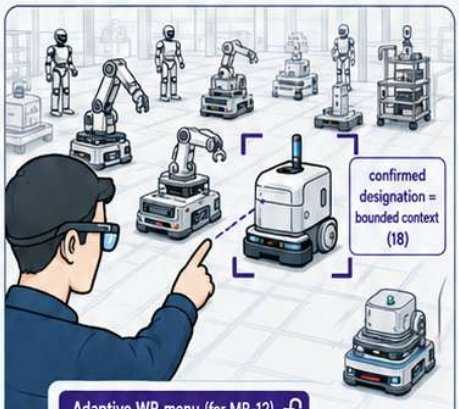
SIGNED CATALOG

- UUMANOID UH-01
- MOBILE ROBOT MR-12
- MANIPULATOR AR-07
- FORKLIFT/AGV AG-03
- INSPECTION UNIT IU-09

one hall = one Scope · signed capability catalog · every unit declared separately · declared before first scan

The bottleneck shifts from interface diversity to governed designation.

Point → Load → Command



Adaptive WR menu (for MR-12)

pause	R
resume	R
send to dock	C2
release next job	C2
hand over payload	C3

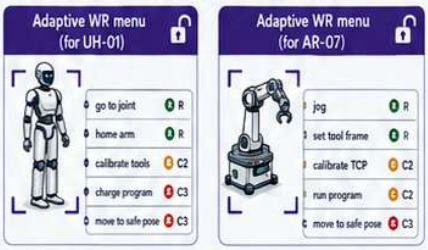
Origin / Consequence: R Routine · C2 Moderate · C3 High

Context & permission scope (for 18)

context loaded · permissions scope · catalog entry · automation chain refs · hash-verified

menu = filtered catalog view — never a free surface

Different unit → different governed menu



automation chains load by reference, hash-verified, on selection only

Consequence Discipline

Consent & consequence levels

1. routine command — declared class
   examples: pause, resume
2. consequential command — C2 / C3
   requires hardware-backed consent
3. Execution Provider finalizes
   - witnessed transition
   - sealed receipt
   - one evidence chain across all units

multi-system scenarios = sequences of designations under one evidence chain

evidence chain can span mobile robots, manipulators, and humanoids

Safety boundary

E-stop and interlocks remain the plant safety system

WR commands act only within declared capabilities.

- ✓ operator never types addresses
- ✓ selected unit loads its own governed menu
- ✓ one evidence chain across multiple units (many types)
- ✗ no free-form control surface
- ✗ no undeclared robot commands
- ✗ no bypass of plant safety systems

The Governed Designation Flow (one unit at a time)

Enter ID2 → Point designate one unit → Confirm frame accepted → Load catalog entry + scope → Preview class and effect shown → Consent C2 / C3 where required → Finalize local execution → Receipt verifiable record

What This Guarantees

- designation instead of addressing
- selection-scoped loading
- member-specific adaptive menu
- hash-verified automation references

What This Explicitly Excludes

- manual address targeting
- undeclared commands
- hidden safety bypass
- free-surface automation

The operator never addresses; the operator designates.

From Store Shelf to Living Room — The Same Contract

The electronics store, the smart home, and the workstation — one architecture, different consequence profiles.

1 The Electronics Store (ID0)

Selected product

Smart Watch X1

- 1.43" AMOLED display
- Heart rate, SpO_2, GPS
- 7 + day battery
- 5 ATM water resistant

Live price: €249.00
In stock: 13 units

Adaptive WR Menu (member)

Compare | Reserve | Order

- Point or scan the shelf tag — exactly this product loads.
- Specs, live price & stock via declared slots.
- Adaptive WR menu: compare - reserve - order.
- **The customer never loads the store; the customer loads what interests them.**

Browsing as silent as walking the aisles — read path local, no undeclared telemetry.

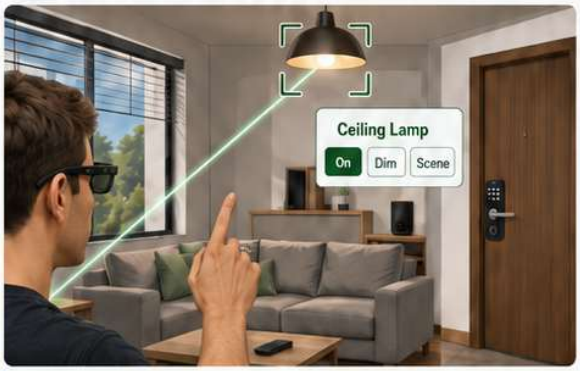
Identity rises only where consequence begins (order — declared floor, login-bound by reference).

2 Continue at the Workstation

- The same handshake-bound session can be reopened or continued at the workstation / WR Desk.
- More screen real estate for comparisons, session history, selected items, and permitted actions.
- Not a new session — a continuation of the same governed context.
- Adaptive WR menu remains context-aware and member-specific.

Same context | Same permissions | Same history

3 The Smart Home



Dim the light — C0 / C1
routine, low-impact command

Open the gate / unlock the door — higher class, steps up to stronger consent (C2 / C3).

A difference of declaration, not of trust in the moment.

Guests: same Scope, tighter inherited constraints — the ratchet as a household feature.

Local mapping (inside the home)

Semantic command
dim / turn on light
↓
Execution Provider
(Home Assistant, Node-RED, OpenHAB, ioBroker, KNX, ...)
↓
Actual device
light

- Semantic command is mapped locally by the Execution Provider.
- Addresses, integrations, and topology stay local.
- No cloud mediation. Everything resolves locally.

1 Scan at the door (ID0)	2 Browse (silent)	3 Point at product	4 Load (only this)	5 Adaptive WR menu	6 Continue at workstation / compare	7 Point at home device	8 Receipt / history
Automatic handshake with the site / home Scope	Catalog appears locally, filtered by declaration	Designation creates bounded context (B).	Specs, status, actions, context, permissions.	Compare - Reserve - Order context-aware actions	More screen real estate, history, and comparisons.	Designation carries the same contract into the home.	Signed receipt and anchored history, verifiable later.

One scan opens a governed world — in the store, at home, and on the workstation.

Scoped Worlds — One Scan Opens a Governed World

1 / 6

One entry, one Scope. Objects load on selection — never wholesale, never silently observed.

1 Enter one governed Scope

Entry (one scan)

Shop | Hall | Home

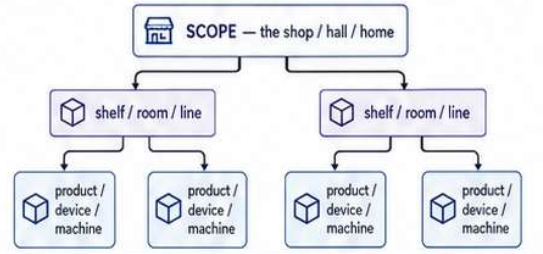
Scope Card

SCOPE
— the shop / hall / home

Scope ID	sc-7F9A-2C3D
Publisher	Acme Retail Ltd.
Root Signature	SIG-7E8B…42D1
Issued	2025-05-07 10:12Z
Depth	0 (root)

Valid & Verified

Scope Graph (example)



publisher-defined — never operator-restructured

one entry = one Scope

single object = Scope of depth zero

ratchet inheritance: children may only tighten

multi-membership → intersection of permissions

2 Signed capability catalog

declared per member, signed under the publisher root, hash-identified

Smart Light A19

Commands: pause | reserve | dim

typed params	yes
consequence class	low
identity floor	member
transition class	reversible
Member ID	m-11A3-9B2E…
Catalog Hash	H-90AF…C312

Conveyor Line 3

Commands: pause | inspect | start

typed params	yes
consequence class	medium
identity floor	member
transition class	controlled
Member ID	m-22D4-7C5F…
Catalog Hash	H-B712…E44D

Shelf Item B12

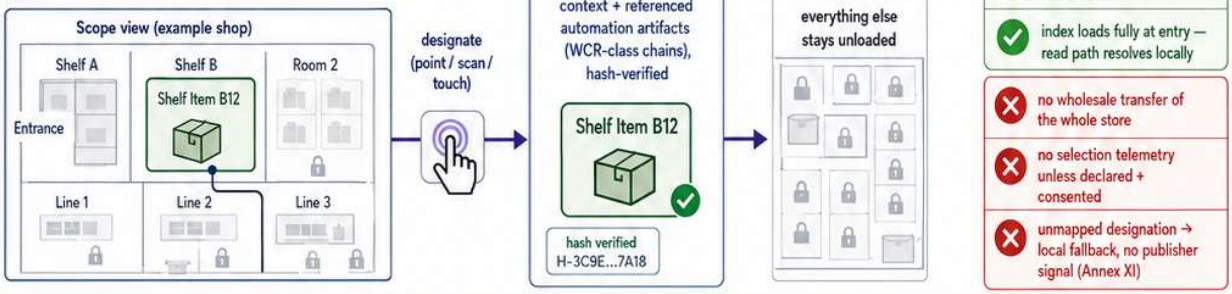
Commands: reserve | locate | compare

typed params	yes
consequence class	low
identity floor	member
transition class	reversible
Member ID	m-33F6-1D8A…
Catalog Hash	H-3C9E…7A18

Nothing shown at runtime is composed at runtime — every command exists in the catalog before the first scan.

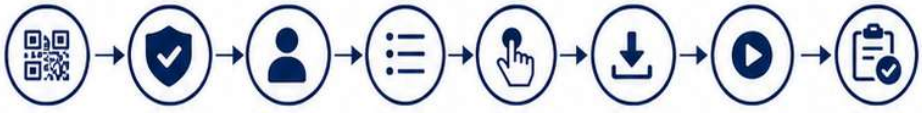
menu = context-filtered view of the catalog — *never* a free surface

3 Selection-scoped loading



- load boundary = the declaration
- index loads fully at entry — read path resolves locally
- no wholesale transfer of the whole store
- no selection telemetry unless declared + consented
- unmapped designation → local fallback, no publisher signal (Annex XI)

4 Local browse → selected load → declared invocation



Scan — enter the Scope
Verify — signature & integrity
Consent — operator consent
Browse — local index
Select — choose one member
Load — hash-verified artifacts
Invoke — issue declared commands
Receipt — verifiable local receipt

What This Guarantees

- one entry, one Scope
- only selected member loads
- verifiable local execution

What This Explicitly Excludes

- silent observation
- bulk preloading
- undeclared surfaces

The operator receives only what they selected; the publisher delivers only what they declared.